

DAY

05

Computer Architecture, OS Internals & Assembly Intro

Foundations for Reverse Engineering , Malware & Exploit Development

CPU & MEMORY

OS INTERNALS

x86 / x64 ASSEMBLY

How Day 05 Is Organized

01



Architecture & OS Internals

Registers, memory hierarchy, stack/heap, endianness, processes, virtual memory, kernel mode, the Registry

02



x86/x64 Assembly Intro

Core instructions, calling conventions, Intel vs AT&T syntax, PE vs ELF at a high level

03



Debugger Trace

Trace a tiny program's assembly; identify the stack, registers, and a function prologue/epilogue

04



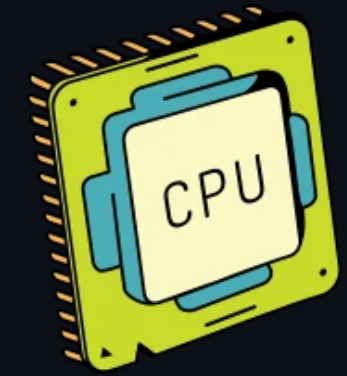
CHECKPOINT

Annotate Disassembly

Label registers, the stack, and control flow on a short disassembly listing

What is CPU ?

CENTRAL PROCESSING UNIT



The primary hardware component of a computer that executes program instructions — performing the arithmetic, logic, control, and input/output operations that make software run. Often called the computer's "brain."

Operational Cycle of CPU

FETCH

Program Counter

The CPU is the processor that carries out a program's instructions. Each cycle it reads the next instruction from memory at the address held in the program counter

DECODE

Control Unit

The control unit interprets that instruction and works out which operation it calls for and which operands it needs.

EXECUTE

ALU

The arithmetic logic unit carries out the operation itself — arithmetic, logic, or a memory-address calculation.

STORE

Registers

The result is written back to a register (or memory), and the cycle repeats for the next instruction — billions of times a second.



CPU Registers

● General Purpose

RAX / EAX
Accumulator — arithmetic, return value

RBX / EBX
Base — general storage

RCX / ECX
Counter — loop / shift count

RDX / EDX
Data — I/O, arithmetic extension

RSI / ESI
Source index (string/array ops)

RDI / EDI
Destination index

RBP / EBP
Base pointer — frame base

RSP / ESP
Stack pointer — top of stack

R8 – R15
x64 extra general-purpose regs

● Segment

CS
Code segment — points at running code

DS
Data segment — default data area

SS
Stack segment — the stack region

ES
Extra segment — string ops dest.

FS
Extra — often Thread Env. Block

GS
Extra — kernel/TLS on x64

● Flags (EFLAGS)

ZF
Zero — result was 0

CF
Carry — unsigned overflow

SF
Sign — result negative

OF
Overflow — signed overflow

PF
Parity — low byte even parity

TF
Trap — single-step for debuggers

What is Memory ?



RAM, CACHE & REGISTER

The hardware that stores data and instructions so the CPU can access and work with them — some of it temporary and cleared on shutdown, some of it permanent.

Characterstics of memory

DATA & CODE

Instructions + Values

Memory is where a running program lives — both its instructions and the data those instructions read and write while the CPU works.

ADDRESS

Numeric Address

Every byte has a unique numeric address. The CPU reads or writes a location by referencing that address, often through a pointer.

VOLATILE

RAM vs Disk

RAM is volatile — its contents vanish when power is lost. Disk storage is persistent and keeps data even when the machine is off.

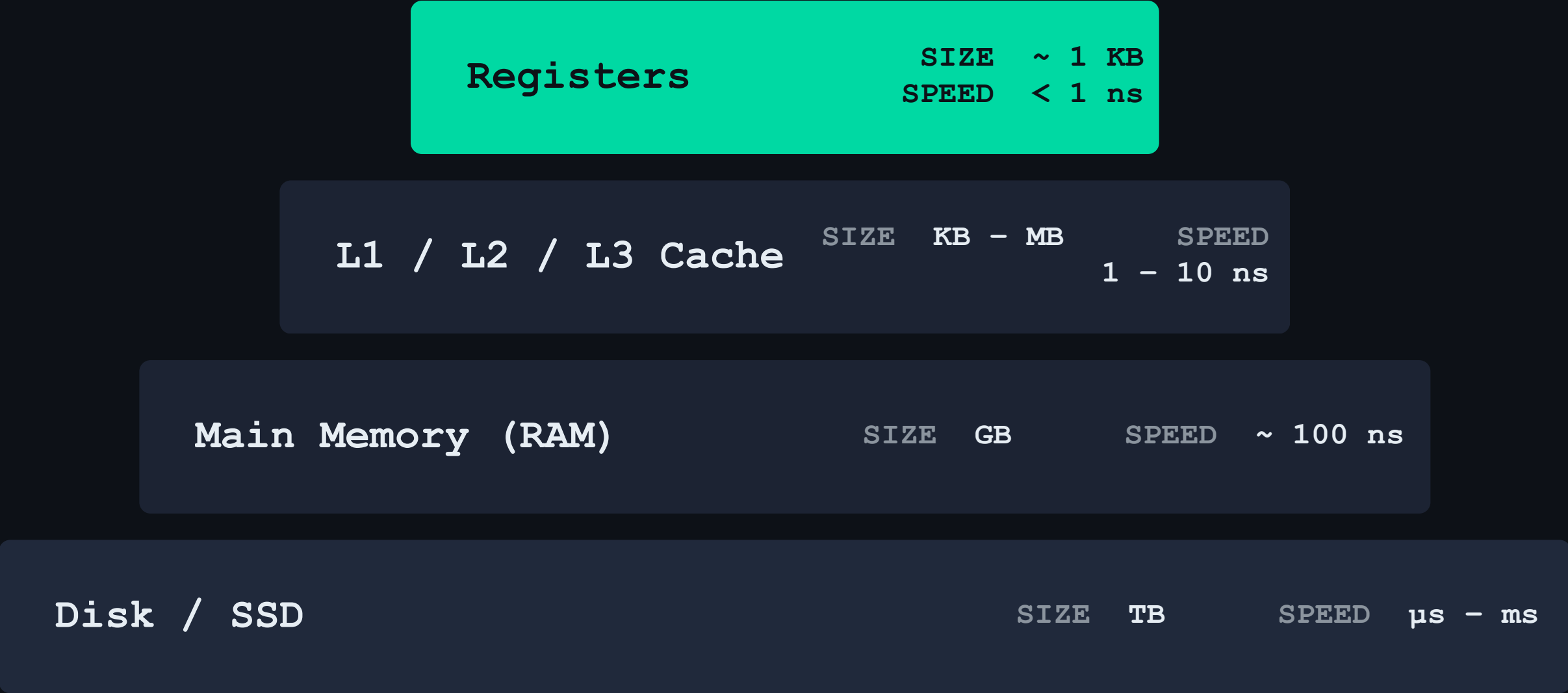
LAYERED

Registers → Disk

Memory isn't just one thing — registers, cache, RAM, and disk form a hierarchy trading speed for size, covered next.



Memory Hierarchy & Addressing

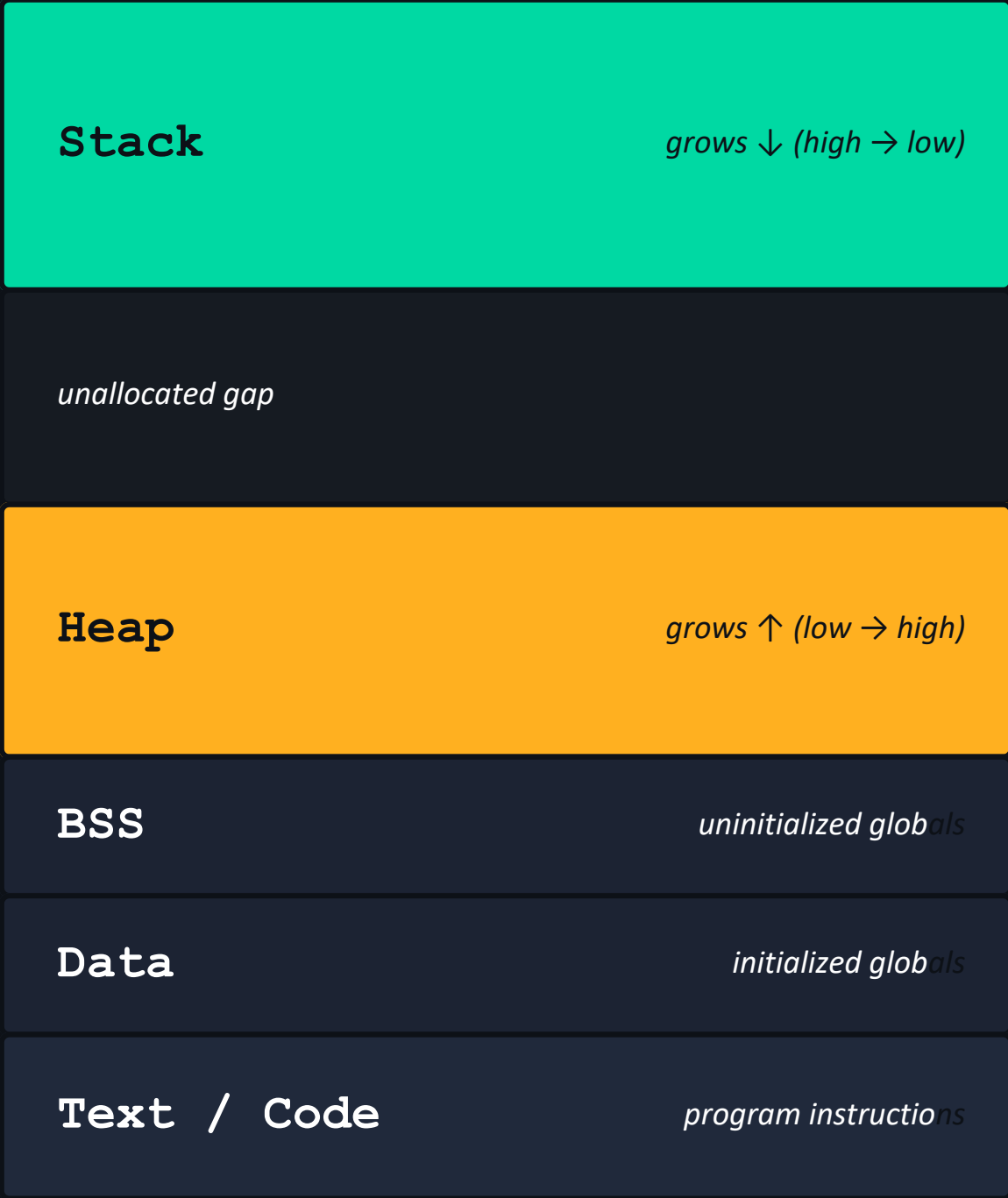


Faster & smaller → slower & larger, top to bottom. The CPU addresses memory by numeric address; a pointer is a register or memory cell holding one. Addressing modes (direct, indirect, base+offset, indexed) determine how an instruction computes that address before a fetch.



Stack & Heap

PROCESS ADDRESS SPACE



STACK vs HEAP

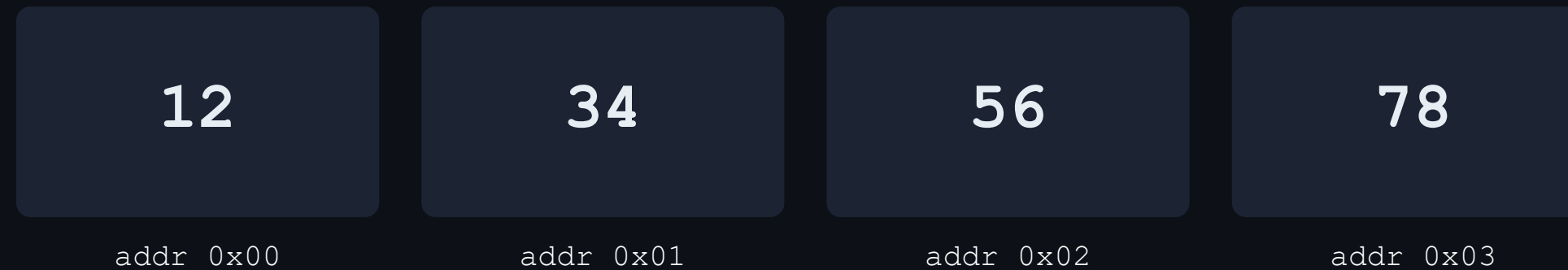
	STACK	HEAP
Allocation	Automatic — grows on CALL / locals	Manual (malloc/free) or GC-managed
Speed	Very fast — pointer bump	Slower — allocator bookkeeping
Size	Fixed at thread creation, limited	Large, bounded mainly by RAM
Lifetime	Freed automatically on return	Lives until explicitly freed
Layout	LIFO — last in, first out	Unordered, can fragment
Typical use	Local variables, return addrs, saved registers, function frames	Dynamic / long-lived data, large or runtime-sized objects



Endianness

How multi-byte values are ordered in memory. Example value: **0x12345678**

Big-Endian



Little-Endian



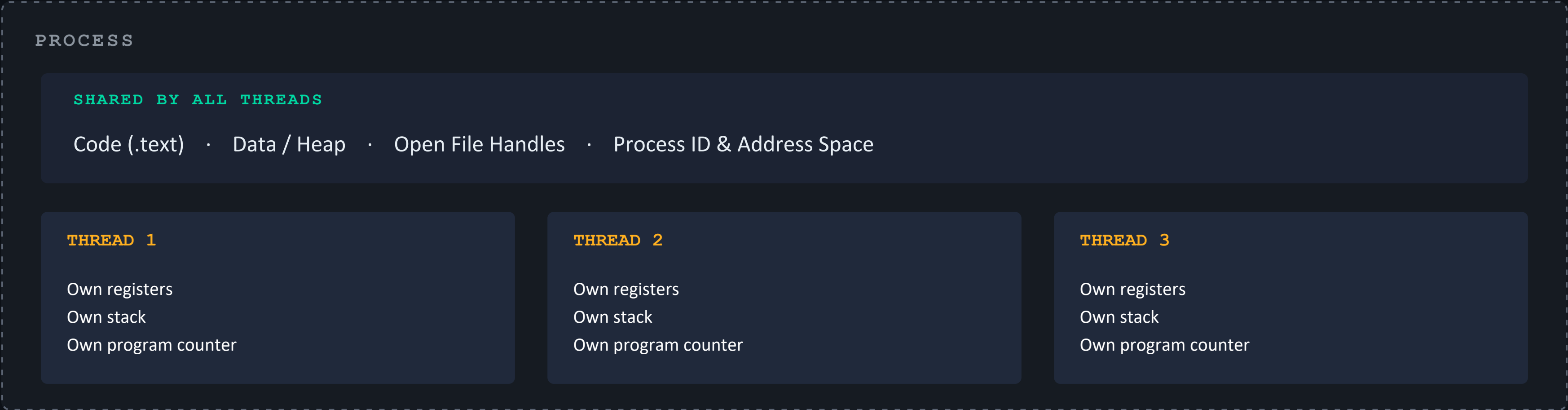
Big-endian stores the most-significant byte first (lowest address) — human reading order. Little-endian stores the least-significant byte first.



x86 / x64 is little-endian. Network protocols (TCP/IP) use big-endian — this is why socket code calls it "network byte order" and often needs `htons()/ntohs()` conversions.



Process & Thread Models



● Process

Isolated address space — a crash or fault stays contained. Communication between processes needs IPC (pipes, sockets, shared memory). Heavier to create and context-switch.

● Thread

Shares its parent process's memory and handles; only registers and the stack are private. Cheaper to create and switch between; a bad thread can corrupt the whole process.

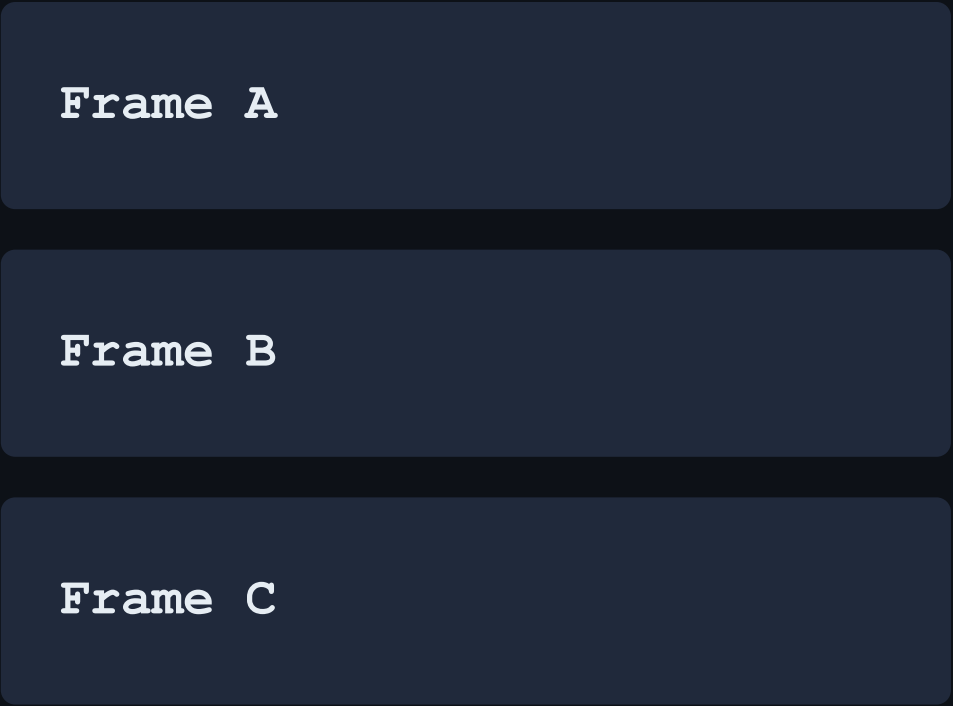


Virtual Memory & Paging

VIRTUAL ADDRESS SPACE



PHYSICAL MEMORY (RAM)



Disk (swap) — page fault

Each process sees a private, contiguous virtual address space. The MMU translates virtual pages to physical frames on every access; an unmapped page triggers a page fault, and the OS pulls it back in from disk.



User Mode, Kernel Mode & System Calls



USER MODE · RING 3

Applications, libraries (libc), and the GUI run here with restricted privileges — no direct hardware or kernel-memory access.

SYSCALL



KERNEL MODE · RING 0

The OS core, drivers, and the scheduler run here with full hardware access — memory, disk, and CPU control.

A SYSCALL, STEP BY STEP

- 1 App calls a library wrapper, e.g. printf()
- 2 Wrapper issues a syscall / int 0x80 instruction
- 3 CPU traps into the kernel; mode switches to Ring 0
- 4 Kernel validates args & performs the operation
- 5 Kernel returns (sysret); mode switches back to Ring 3



The Windows Registry

A hierarchical database of keys, subkeys, and values that stores OS and application configuration — organized into five root hives.

HKLM HKEY_LOCAL_MACHINE System-wide hardware, driver & software config	HKCU HKEY_CURRENT_USER Settings for the currently logged-in user	HKCR HKEY_CLASSES_ROOT File associations & COM class registrations	HKU HKEY_USERS Loaded profiles for every user on the machine	HKCC HKEY_CURRENT_CONFIG Active hardware profile at boot
---	---	---	---	---



WHY IT MATTERS FOR SECURITY WORK

The Registry is a major forensic and malware-analysis artifact: startup persistence commonly hides in Run / RunOnce keys, installed software and recent activity leave traces under HKCU, and services & drivers are registered under HKLM\SYSTEM\CurrentControlSet.



x86 / x64 Assembly Intro

Core instructions, calling conventions, Intel vs AT&T syntax, and the PE / ELF executable formats.

MOV · PUSH · POP · LEA

ADD/SUB/XOR · CMP/TEST

JMP · CALL · RET

cdecl / stdcall / fastcall



Moving Data: MOV, PUSH, POP, LEA

MOV

MOV EAX, EBX

Copies src into dst. Never memory-to-memory — one side must be a register or immediate.

PUSH

PUSH EAX

RSP -= 8 (or 4); stores the value at the new top of stack.

POP

POP EBX

Loads the value at [RSP] into dst; RSP += 8 (or 4).

LEA

LEA EAX, [EBX+4]

Loads the computed address itself — it never dereferences memory. Also used as a fast add/multiply trick.



Arithmetic, Logic & Comparison

ADD**ADD EAX, 5**

dst = dst + src. Updates ZF, CF, SF, OF.

SUB**SUB EAX, ECX**

dst = dst – src. Updates the same flags as ADD.

XOR**XOR EAX, EAX**

dst = dst ^ src. The idiom XOR EAX,EAX is the fast, common way to zero a register.

CMP**CMP EAX, 10**

Computes op1 – op2 to set flags only — the result itself is discarded.

TEST**TEST EAX, EAX**

Computes op1 & op2 to set flags only — commonly used to check for zero or sign.



Control Flow: JMP, JNE, CALL, RET

JMP

Unconditional jump — always taken.

JNE / JNZ

Jump if ZF = 0 (not equal / not zero). Conditional — follows a CMP or TEST.

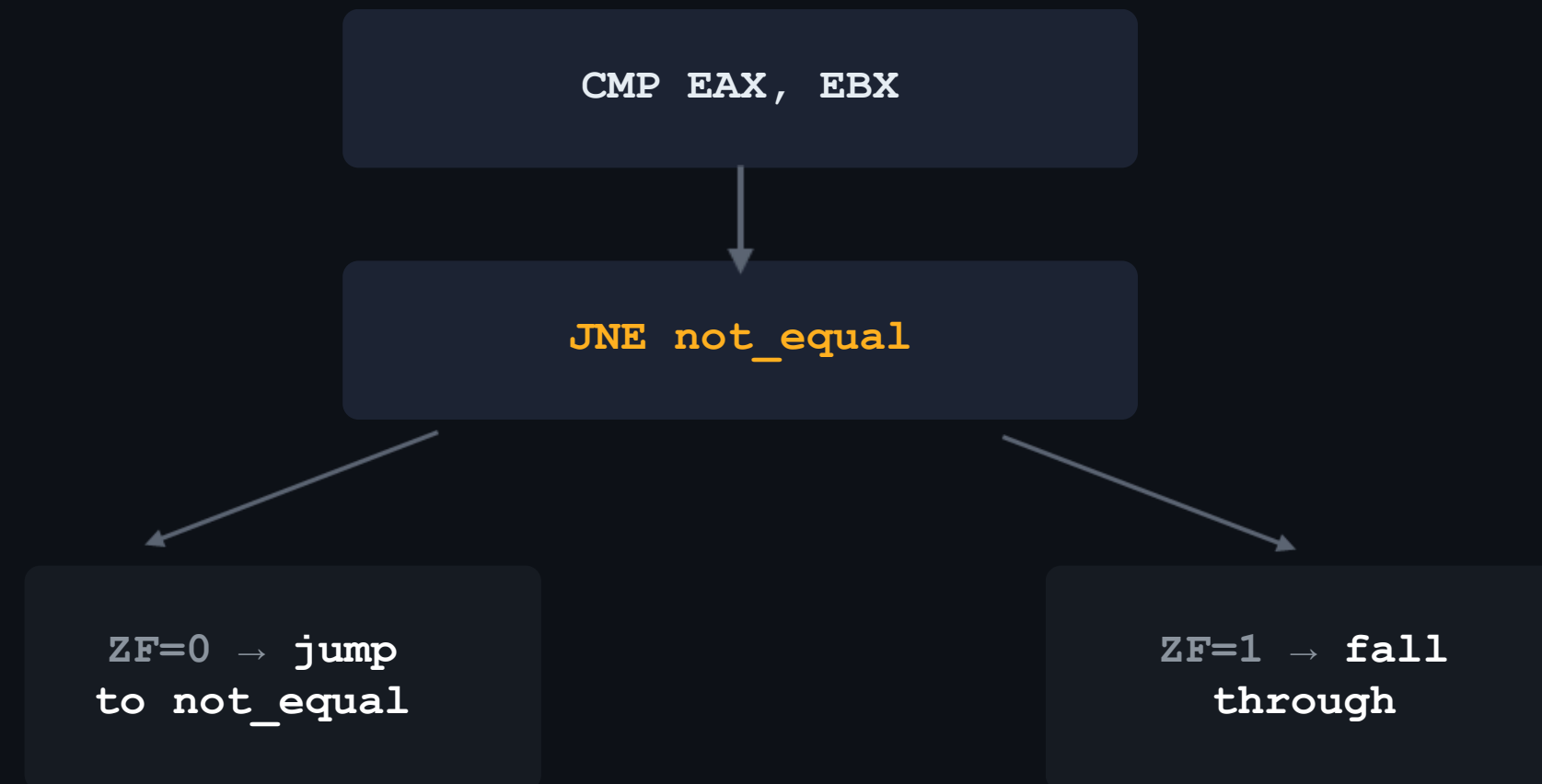
CALL

Pushes the return address, then jumps to func.

RET

Pops the return address off the stack into RIP/EIP.

A CONDITIONAL BRANCH



JNE reads the flags CMP just set — the two instructions are almost always used as a pair.



Calling Conventions

	cdecl	stdcall	fastcall
Argument order	Pushed right-to-left	Pushed right-to-left	First args in registers, rest on stack
Stack cleanup	Caller cleans up	Callee cleans up (RET N)	Callee cleans up
Register use	None reserved for args	None reserved for args	ECX/EDX (x86) or RCX,RDX, R8,R9 (x64) for first args
Return value	EAX / RAX	EAX / RAX	EAX / RAX
Typical use	Default C convention; supports variadic functions	Classic WinAPI functions	Faster for small argument counts; common in x64 ABIs



Intel vs AT&T Syntax

INTEL SYNTAX

```
mov  eax, ebx
mov  eax, [ebx+4]
add  eax, 10
mov  DWORD PTR [eax], 1
```

- dst, src order
- no register prefix
- [] for memory access
- size is a keyword (DWORD PTR)

AT&T SYNTAX

```
movl  %ebx, %eax
movl  4(%ebx), %eax
addl  $10, %eax
movl  $1, (%eax)
```

- src, dst order — reversed
- % prefix on registers
- \$ prefix on immediates
- size suffix on mnemonic (b/w/l/q)

Same instruction, two dialects. Linux tools (GDB, objdump) often default to AT&T; Windows tools (x64dbg, IDA, WinDbg) default to Intel — both are switchable.



PE (Windows) vs ELF (Linux)

PE — Portable Executable

`.exe / .dll` · Windows

- DOS Header & Stub
- PE Header (COFF)
- Optional Header (entry point, subsystem)
- Section Table
- Sections: `.text` `.data` `.rdata` `.rsrc`
- Import / Export Tables (IAT / EAT)

ELF — Executable & Linkable Format

`no extension` · Linux

- ELF Header (magic, class, entry point)
- Program Header Table (segments)
- Section Header Table
- Sections: `.text` `.data` `.bss` `.rodata`
- Dynamic Symbol Table
- `.dynamic` / `.dynsym` / `.dynstr`



Function Prologue & Epilogue

```
push    ebp           ; save caller's base pointer
```

```
mov     ebp, esp      ; set up this frame's base
```

```
sub     esp, 0x10     ; allocate space for locals
```

```
...           ; function body
```

```
mov     esp, ebp      ; deallocate locals
```

```
pop     ebp           ; restore caller's base pointer
```

```
ret                     ; pop return address, jump back
```

PROLOGUE

Saves the caller's frame, then establishes this function's own stack frame and reserves room for its local variables.

BODY

The function's actual instructions — the part that does the real work.

EPILOGUE

Tears the frame back down and hands control back to the caller via RET.



Trace a Tiny Program in a Debugger

- 1 Load a small compiled binary in a debugger (x64dbg, GDB, or WinDbg)
- 2 Step through its assembly one instruction at a time
- 3 Watch the stack pointer and base pointer change on each PUSH/CALL
- 4 Locate the function's prologue and epilogue boundaries



Annotate a Disassembly Listing

Given a short disassembly listing, label the registers in use, every stack operation, and the control-flow path the code takes.

YOUR CHECKLIST

-  Mark where the function prologue ends and the epilogue begins
-  Label each register's role at its first use (argument, counter, return value...)
-  Flag every stack operation — PUSH, POP, and any direct SUB/ADD on ESP/RSP
-  Trace each JMP/CALL and note the flag or condition driving every branch
-  Note where and how control finally returns to the caller

ASM

```
push    ebp
mov     ebp, esp
sub     esp, 0x8
mov     [ebp-0x4], 0
cmp     [ebp-0x4], 0xa
jge     loop_end
add     [ebp-0x4], 1
jmp     loop_start
loop_end:
mov     esp, ebp
pop     ebp
ret
```

Key Takeaways



Registers

General, segment & flag registers drive every instruction



Memory hierarchy

Registers → cache → RAM → disk, fastest to slowest



Stack & heap

LIFO auto-managed frames vs. manually allocated storage



Endianness

x86/x64 is little-endian; network order is big-endian



Processes & threads

Isolated address space vs. shared memory, own stack



Virtual memory

The MMU maps virtual pages to physical frames



User / kernel mode

Ring 3 apps reach Ring 0 only through a syscall



The Registry

Five hives store OS, app & persistence configuration



Core instructions

MOV/PUSH/POP/LEA, ADD/SUB/XOR, CMP/TEST, JMP/CALL/RET



Calling conventions

cdecl, stdcall, and fastcall differ in cleanup & args



Intel vs AT&T

Reversed operand order; % and \$ prefixes in AT&T



PE vs ELF

Windows and Linux structure executables differently

Next: put it into practice in the debugger, then bring your annotated listing to the checkpoint.